

SAP Firefighter Log Compliance Reviewer – Approach

A controller uploads a session JSON file through the web UI. The file is sent via HTTP POST to a FastAPI backend running in Docker, behind a nginx reverse proxy. The backend validates the JSON structure using Pydantic before anything else runs.

The validated session data is passed to the rule engine, which runs 14 deterministic rules against the full parsed session object. Each rule checks a specific binary condition, whether a dangerous tcode is present, whether change volume exceeds the stated reason, whether the firefighter raised their own ticket. Rules are grouped by category: dangerous actions (R-003, R-004, R-005), access control (R-008, R-010), volume and timing (R-006, R-007, R-009), reason quality (R-001, R-002), and four extended rules (R-011, R-012, R-013, R-016) identified by analyzing the 50 labeled training sessions. Importantly, every rule runs on the complete unsampled data, no log entry is ever skipped at this stage. The engine returns all findings sorted by severity.

If any of the five hard rules fired (R-003, R-004, R-005, R-008, R-010), the pipeline returns REJECT immediately with 99% confidence and skips the LLM entirely. These rules detect unambiguous violations where no semantic judgment is needed. This applies to approximately 30% of the sessions in practice, reducing both cost and latency on the most clear-cut cases.

For all other sessions, the pipeline calls Claude Haiku via the OpenAI-compatible API. To reduce token consumption and cost, the session is flattened into a compact text summary before the call, neutral diagnostic tcodes (SE80, SU53, /NEX) are excluded since they appear in every session and add no signal, and empty log sections are omitted entirely. For sessions with large change logs, entries are sampled across beginning, middle and end rather than sent in full. This sampling is safe because the deterministic rules have already scanned every entry, the LLM does not need to audit individual changes, it needs to understand the pattern of activity, which a representative sample captures equally well. This reduces average input tokens from roughly 1,200 to around 450 per session.

The deterministic findings are included in the LLM prompt as context so the LLM focuses on what the rules could not judge: whether the stated reason actually matches the actions in spirit, whether the justification is specific enough, and what the controller should ask the firefighter to clarify. The LLM returns a structured JSON verdict with semantic findings and an optional suggested correction. LLM findings are deduplicated against deterministic ones by rule ID and location before merging. If the LLM fails for any reason, the pipeline falls back to a deterministic-only verdict with 0.5 confidence so the system never crashes.

The result is saved to a SQLite database and returned to the frontend. The controller sees the verdict banner, color-coded findings with exact evidence from the log, and a suggested correction message if the session needs clarification. They click Pass, Reject, or Send Back to record their final decision.

The same pipeline function is used by both the API and the eval harness, so UI results and evaluation metrics are always consistent.

Oskar Klos